

Abdullah Subhaan K

Presenter Pack

Member C — Interface, Output & Testing

Slide 3

Maze Runner · Problem Statement 15

REVA University · Hackathon अभिनव · 07/10/2026

SECTION 1 — The project (everyone must know this)

What we built

A maze game where the player has to reach the exit while an enemy chases them using real pathfinding. The enemy recalculates the shortest route to the player on every single step, using either BFS or A*, and you can switch between the two live and watch the difference on screen.

From level 4 a **second enemy** joins running the *other* algorithm, so BFS and A* hunt the same player at the same time.

How it works, in 60 seconds

The maze is a grid of 1s and 0s — 1 is a wall, 0 is floor. The player and each enemy sit on one cell. Sixty times a second we read the keyboard, move everything one cell if its cooldown has expired, and redraw. The enemy decides where to go by running a search from itself to the player, then stepping onto the second cell of the route it found.

Everything else is detail on top of that.

The five facts everyone must be able to say

1. **BFS uses a FIFO queue, so it finds the shortest path.** It finishes every cell at distance d before touching anything at $d+1$, so the first time it reaches the goal it got there in the fewest steps.
2. **A* scores cells** $f = g + h$ — g is steps taken, h is Manhattan distance remaining. h must never overestimate or optimality is lost.
3. **A* expanded ~26% fewer cells but ran slower** in wall-clock time, because its heap costs $O(\log n)$ per operation against the deque's $O(1)$.
4. **The enemy used to camp the exit** — "farthest from the player" is the opposite corner, which is where the exit is. Fixed by maximising `min(distance to player, distance to exit)`.
5. **We tuned difficulty by measurement, not feel** — `autoplay.py` ran 500 bot playthroughs and found the enemy was only $1.1\times$ slower than the player.

The numbers

Tech	Python 3 + pygame-ce. No dataset, no network, no hardware
Tests	226 headless checks, all passing
Benchmark	50 mazes per configuration
Balance	500 bot playthroughs; L1–3 ~68–70%, L4–10 20–38%
Repo	github.com/Hyperval/maze-runner

Who built what

Person	Owns
Akhil Sathish Kumar	Maze generation, braiding, difficulty balance
Akshay N	Pathfinding — BFS, A*, distance maps
Abdullah Subhaan K	Rendering, menu, pause, HUD, test suite
Christen Mendis	benchmark.py, autoplay.py, docs, demo video

SECTION 2 — Your code: rendering, state machine, tests

You own everything the judges actually see, plus `test_pathfinding.py`.

The game loop

```
while True:
    self.handle_events() # read keyboard and window events
    self.update()        # move everything, decide win/loss
    self.draw()          # paint the frame
    self.clock.tick(FPS) # sleep so we run at 60fps, not 5000fps
```

Every real-time game is this loop. Everything else hangs off it.

The state machine

```
MENU, PLAYING, PAUSED, WON, LOST = "menu", "playing", "paused", "won", "lost"
```

`self.state` holds exactly one of these. `update()` returns immediately unless the state is `PLAYING` — that's what makes pausing work. Nothing moves, but `draw()` keeps running, so the frozen frame stays on screen.

Using strings rather than numbers means debug output reads `"playing"`, not `2`.

The pause clock — a subtle bug you fixed

`pygame.time.get_ticks()` keeps counting while paused. If you ignored that, the timer would jump forward by the pause duration the moment you resumed. So we record when the pause began, add the duration to `paused_total`, and subtract it:

```
self.elapsed = (now - self.start_time - self.paused_total) / 1000.0
```

Good one to mention if asked "what was fiddly?"

Why movement is timed in milliseconds, not frames

```
if now - self._last_move < self.move_delay:
    return False          # cooldown still running, refuse the move
```

If we counted *frames*, the game would run faster on a better laptop. Timing in milliseconds makes it behave identically on every machine.

The search overlay — your most visible work

For each enemy we draw every cell its search expanded as a translucent rectangle, then its chosen path as a line. Two enemies means two coloured regions at once.

Design decision worth stating: A*'s region is orange, not cyan. The player marker is cyan, so a cyan search region around the player would be unreadable. Colours that must be distinguished differ in lightness, not just hue.

Why the window is a fixed size

`WIDTH` / `HEIGHT` are computed once from `MAX_COLS` / `MAX_ROWS`. Levels grow the maze but it's capped at those maxima, and smaller mazes are centred using `offset_x` / `offset_y`. If the maze could outgrow the window it would be drawn off-screen.

The test suite — 226 checks

Run after **every** change: `python test_pathfinding.py`

The checks that matter most:

Check	The bug it guards against
Every cell reachable, 20 mazes	A maze where the enemy can't reach you
A* path length == BFS path length	A broken heuristic
A* expands $\leq$ BFS	A heuristic that makes things worse
Enemy > 8 steps from exit	The exit-camping bug
<code>ENEMY_MIN_DELAY > PLAYER_MOVE_DELAY</code>	Enemy becoming strictly faster than you
300 random moves + rendering	Any crash from an unexpected input

That last one is a **fuzz test** — it does random things and only asserts "don't crash". It's the one that finds the bug that would otherwise appear in front of the examiner.

SECTION 3 — Your presenting job

Two jobs: **Slide 3 (Tech stack)**, and **driving the live demo**. The demo is worth more marks than the slide, so treat the slide as a quick stop.

Slide 3 — Tech stack (target: 30 seconds)

Do not read out all seven boxes. Land one point:

Five modules make up the game — pathfinding, maze generation, entities, the main loop and settings. The three along the bottom are different: those are tools we wrote to check our own work. A benchmark that measures the two algorithms, a bot that plays the game hundreds of times looking for crashes, and 226 automated tests.

That's the part we'd point at — we didn't just build it, we built the things that tell us whether it works.

Then move on. If asked who wrote what, the ownership table is in Section 1.

Driving the live demo (3–4 minutes) — your main job

You built the interface, so you play. **Rehearse this twice before presenting.**

Time	Do	Who speaks
0:00	Title screen, press SPACE	You
0:20	Move around, collect a coin	You: "collect every coin to unlock the exit"
0:45	Press V to show the overlay	Akshay takes over narration
1:15	Press TAB to switch algorithms	Akshay: the counts in the HUD
1:45	Reach the exit, win	You: "levels get harder — bigger maze, faster enemy"
2:15	Let level 4+ load, show two enemies	Akshay

Rules while playing: - Play a level you've practised. Don't improvise. - If you're about to die, that's fine — losing on screen is not a failure, it shows the enemy works. Press SPACE and carry on. - Don't narrate your own keystrokes. Let Akshay talk about the algorithm while you play.

If the demo crashes

Say one sentence — *"Let me show you the recorded version"* — and hand to Christen for the backup video.

Do not apologise repeatedly and do not try to debug it live. Having the backup ready is worth more than it never crashing.

Questions that come to you

- *"How does the game loop work?"* — events, update, draw, tick.
- *"Why milliseconds not frames?"* — so it plays the same on a fast and a slow machine.
- *"How do you detect collisions?"* — everything moves one whole cell at a time, so a collision is just checking whether two things are on the same cell. No rectangle overlap maths needed.
- *"How did you test it?"* — 226 headless checks, including a fuzz test that does 300 random moves with rendering on and asserts nothing crashes.
- *"What's the purple area?"* — cells the search expanded. (Akshay can take this one if you'd rather.)

Final checklist

- ☐ Rehearse the demo end to end, twice, exactly as you'll present it
- ☐ Run `python test_pathfinding.py` the morning of — confirm 226 passing
- ☐ Game already open on the title screen before you walk in
- ☐ Know which level you'll play, and have played it before
- ☐ Laptop charged